

shark on wire 1

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

We found this packet capture. Recover the flag.

Hints

- Try using a tool like Wireshark. What are streams?
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read PCAP metadata
 - **Wireshark** – GUI tool for analyzing network traffic
 - `tshark` – Command-line Wireshark tool for packet analysis
 - `sort` – Organize timestamps in chronological order
 - `cut` – Extract specific fields from output
 - `xxd` – Convert hexadecimal to binary data
-

Complete Solution

Step 1: Download and Prepare

Download the PCAP file named `capture.pcap` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file capture.pcap
```

Expected output:

```
capture.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4
(Ethernet, capture length 262144)
```

The file is a standard **PCAP** capture file containing Ethernet traffic.

Step 3: Search for Visible Text

Check if the flag appears as plain text:

```
strings capture.pcap | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible. We need to analyze the network traffic.

Step 4: Examine Metadata

```
exiftool capture.pcap
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : capture.pcap
Directory                   : .
File Size                    : 239 kB
...
File Type                    : PCAP
File Type Extension          : pcap
MIME Type                    : application/vnd.tcpdump.pcap
PCAP Version                 : PCAP 2.4
```

Byte Order : Little-endian (Intel, II)
Link Type : BSD Loopback
Time Stamp : 2019:07:12 22:59:07.657471+02:00

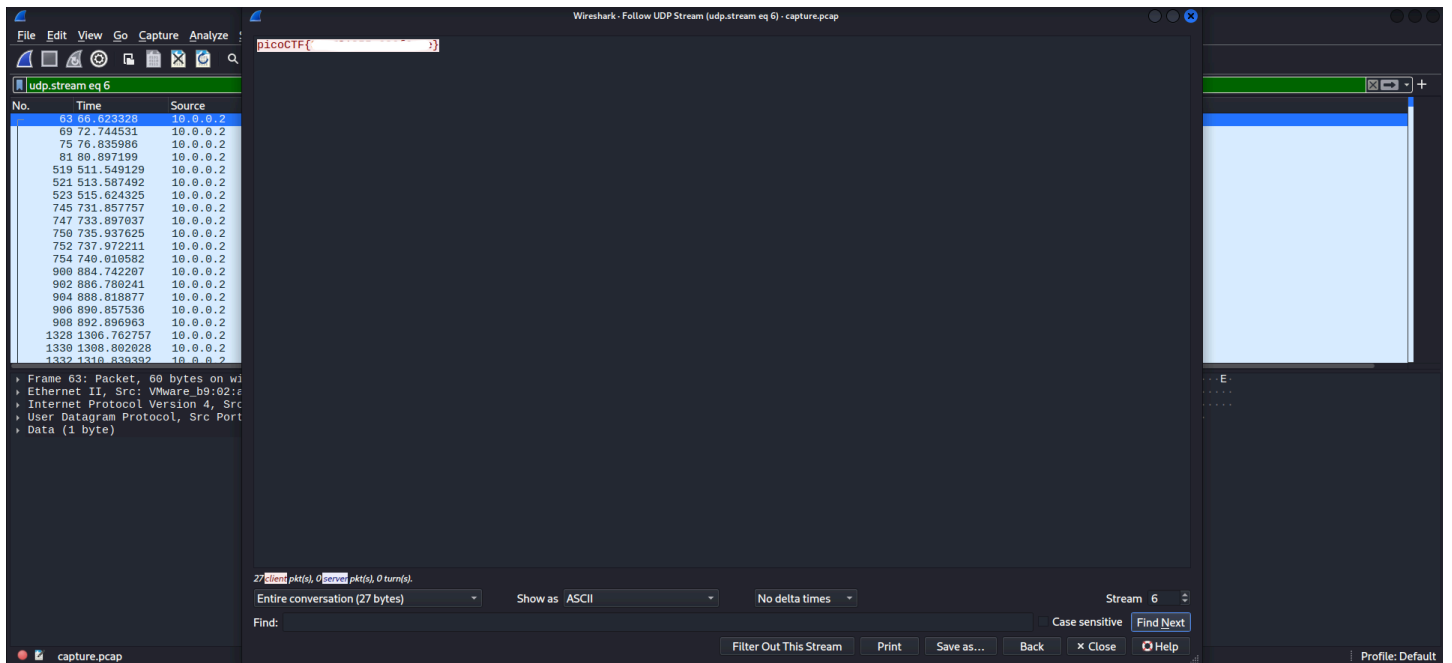
The metadata doesn't reveal the flag.

Step 5: Analyze with Wireshark

Open the file in Wireshark:

The top screenshot shows the Wireshark interface with the 'capture.pcap' file loaded. The packet list on the left shows a series of packets, with packet 13 selected. The packet details pane on the right shows the selected packet's structure: Ethernet II, Internet Protocol Version 4, User Datagram Protocol, and Simple Service Discovery Protocol. The packet bytes pane on the right shows the raw data of the selected packet.

The bottom screenshot shows the Wireshark interface with the 'capture.pcap' file loaded. The packet list on the left shows a series of packets, with packet 63 selected. The packet details pane on the right shows the selected packet's structure: Ethernet II, Internet Protocol Version 4, User Datagram Protocol, and Data (1 byte). The packet bytes pane on the right shows the raw data of the selected packet.



After thorough investigation, a specific **UDP stream** catches our attention: `udp.stream eq 6` .

Observing the packets in this stream, we notice that each packet contains a single hexadecimal value. When converted to ASCII, these values spell out the flag – but the characters are spread across multiple packets.

Step 6: Extract the UDP Stream with tshark

Instead of manually extracting each packet, we'll use `tshark` for automated extraction:

```
tshark -nr capture.pcap -Y "udp.stream eq 6" -T fields -e frame.time -e udp.payload
```

Output:

```
2019-07-12T23:00:14.280799000+0200 70
2019-07-12T23:00:20.402002000+0200 69
2019-07-12T23:00:24.493457000+0200 63
2019-07-12T23:00:28.554670000+0200 6f
2019-07-12T23:07:39.206600000+0200 43
2019-07-12T23:07:41.244963000+0200 54
2019-07-12T23:07:43.281796000+0200 46
2019-07-12T23:11:19.515228000+0200 7b
2019-07-12T23:11:21.554508000+0200 53
2019-07-12T23:11:23.595096000+0200 74
2019-07-12T23:11:25.629682000+0200 61
2019-07-12T23:11:27.668053000+0200 54
2019-07-12T23:13:52.399678000+0200 33
```

2019-07-12T23:13:54.437712000+0200	31
2019-07-12T23:13:56.476348000+0200	33
2019-07-12T23:13:58.515007000+0200	35
2019-07-12T23:14:00.554434000+0200	35
2019-07-12T23:20:54.420228000+0200	5f
2019-07-12T23:20:56.459499000+0200	36
2019-07-12T23:20:58.496863000+0200	33
2019-07-12T23:21:00.537586000+0200	36
2019-07-12T23:21:02.574144000+0200	66
2019-07-12T23:21:04.613179000+0200	36
2019-07-12T23:21:06.650861000+0200	65
2019-07-12T23:21:08.688217000+0200	36
2019-07-12T23:21:10.729138000+0200	65
2019-07-12T23:21:12.772069000+0200	7d

The `udp.payload` field contains hexadecimal values (e.g., `70` , `69` , `6f`). Each value is a **ASCII code** that represents a character.

Step 7: Reconstruct and Decode the Payload

We need to extract just the payload data, sort by timestamp (to ensure correct order), convert hex to ASCII, and display as text:

```
tshark -nr capture.pcap -Y "udp.stream eq 6" -T fields -e frame.time -e udp.payload |
sort | cut -f2 -d '$\t' | xxd -r -p
```

Output:

picoCTF{<REDACTED>}

Understanding the Pipeline

Command	Purpose
<code>tshark -nr capture.pcap -Y "udp.stream eq 6"</code>	Extract UDP stream 6
<code>-T fields -e frame.time -e udp.payload</code>	Output timestamp and payload

Command	Purpose
<code>sort</code>	Sort by timestamp (chronological order)
<code>cut -f2 -d \$'\t'</code>	Extract only the payload column (second field)
<code>xxd -r -p</code>	Convert hex to ASCII (reverse hexdump)

✔

Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, running the command above will reveal the complete flag.

🧠

Key Concepts

Concept	Description
PCAP (Packet Capture)	A file format for capturing network traffic, containing raw packet data.
UDP Stream	A sequence of UDP packets between two endpoints (identified by <code>udp.stream</code>).
tshark	Command-line version of Wireshark, ideal for automated packet analysis.
Hexadecimal Payload	Packet payloads are often displayed in hexadecimal; <code>xxd -r -p</code> converts hex back to text.
ASCII Codes	Each character has a corresponding hexadecimal ASCII value (e.g., <code>70</code> = 'p', <code>69</code> = 'i').

Concept	Description
Stream Sorting	Packets may arrive out of order; sorting by timestamp ensures correct reconstruction.

Pro Tips

- 1. **Use `udp.stream` filters** – This groups packets by conversation, making it easier to reconstruct complete exchanges.
- 2. **`xxd -r -p` is your friend** – It converts hexadecimal strings back to readable text (reverse hexdump).
- 3. **Not all flags are in TCP** – UDP streams can also carry hidden data, as demonstrated here.
- 4. **Sort by timestamp** – Packets can arrive out of order. Always sort by timestamp when reconstructing data split across multiple packets.
- 5. **ASCII values vs raw text** – Sometimes payloads contain raw ASCII codes (`70` = 'p') rather than the characters themselves. Convert hex to ASCII to decode.
- 6. **Pipeline power** – Combining `tshark` , `sort` , `cut` , and `xxd` extracts, reconstructs, and decodes in one go.

Alternative ASCII Decoding Tools

Tool	Command / Link
Linux <code>xxd</code>	<code>xxd -r -p</code>
CyberChef	https://gchq.github.io/CyberChef/ (Operation: From Hex)
Python	<code>bytes.fromhex(hex_string).decode()</code>
Online	https://www.rapidtables.com/convert/number/hex-to-ascii.html